

LC 2463: Minimum Total Distance Traveled

Authored by: Arpit Raj, LNMIIT Jaipur

October 1, 2026

Infosys, Google

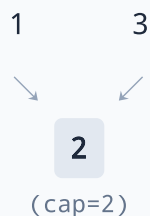
2D Array Definitions

- **robot[i]** → pos of i^{th} robo
- **factory[j]** → pos of j^{th} factory, max robos it can repair

Objective

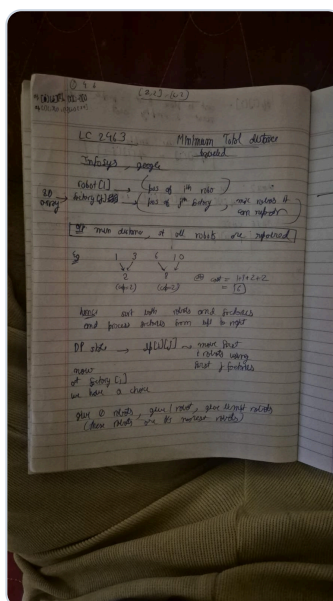
min distance, st all robots are repaired

Example Walkthrough



$$\text{cost} = 1 + 1 + 2 + 2 = 6$$

hence sort both robots and factories and process factories from left to right



Reference Note 1: Problem setup and logic

DP State Formulation

DP state $\rightarrow dp[i][j]$ ~ move first i robots using first j factories

now at factory $[j]$ we have a choice:

- give 0 robots
- give 1 robot
- give limit robots (*these robots are its nearest robots*)

Base Case

$$dp[0][0] = 0$$

there is nothing like 0th factory so no cost

Transitions

now for every factory:

$$dp[i][j+1] = \min(dp[i][j+1], dp[i][j])$$

(not sending any robo to this factory)

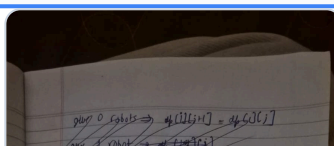
now run a loop to send k robots to this factory

```
cost = 0
for (int k = 1; k <= limit; k++) {
    cost += abs(robot[i+k-1] - pos)
}
```

Eg. let $k=2$ and $i=3$

meaning 3 robot process ho gaye aur do bhejne he, hence $(3+2)=5$
and -1 cause robots is 0 indexed

$$dp[i+k][j+1] = \min(dp[i+k][j+1], dp[i][j] + cost)$$



C++ Implementation

The complete code algorithm translating the DP states formulated in the previous sections.

```
class Solution {
public:
    long long minimumTotalDistance(vector<int>& robot, vector<vector<int>>&
factory) {

        sort(robot.begin(),robot.end());
        sort(factory.begin(),factory.end());

        int n=robot.size();
        int m=factory.size();

        vector<vector<long long>> dp(n+1,vector<long long>(m+1,LLONG_MAX));

        dp[0][0]=0;

        for(int j=0;j<m;j++){
            int pos=factory[j][0];
            int lim=factory[j][1];

            for(int i=0;i<=n;i++){

                //do hi mat
                dp[i][j+1]=min(dp[i][j+1],dp[i][j]);
                long long cost=0;
                for(int k=1;k<=lim && i+k-1<n;k++){
                    cost+=abs(robot[i+k-1]-pos);
                    if(dp[i][j]!=LLONG_MAX){
                        dp[i+k][j+1]=min(dp[i+k][j+1],dp[i][j]+cost);
                    }
                }
            }
        }

        return dp[n][m];

    }
};
```